

Chapter 7: Stacks

Fundamentals of Python: Data Structures, Second Edition

Learning Objectives

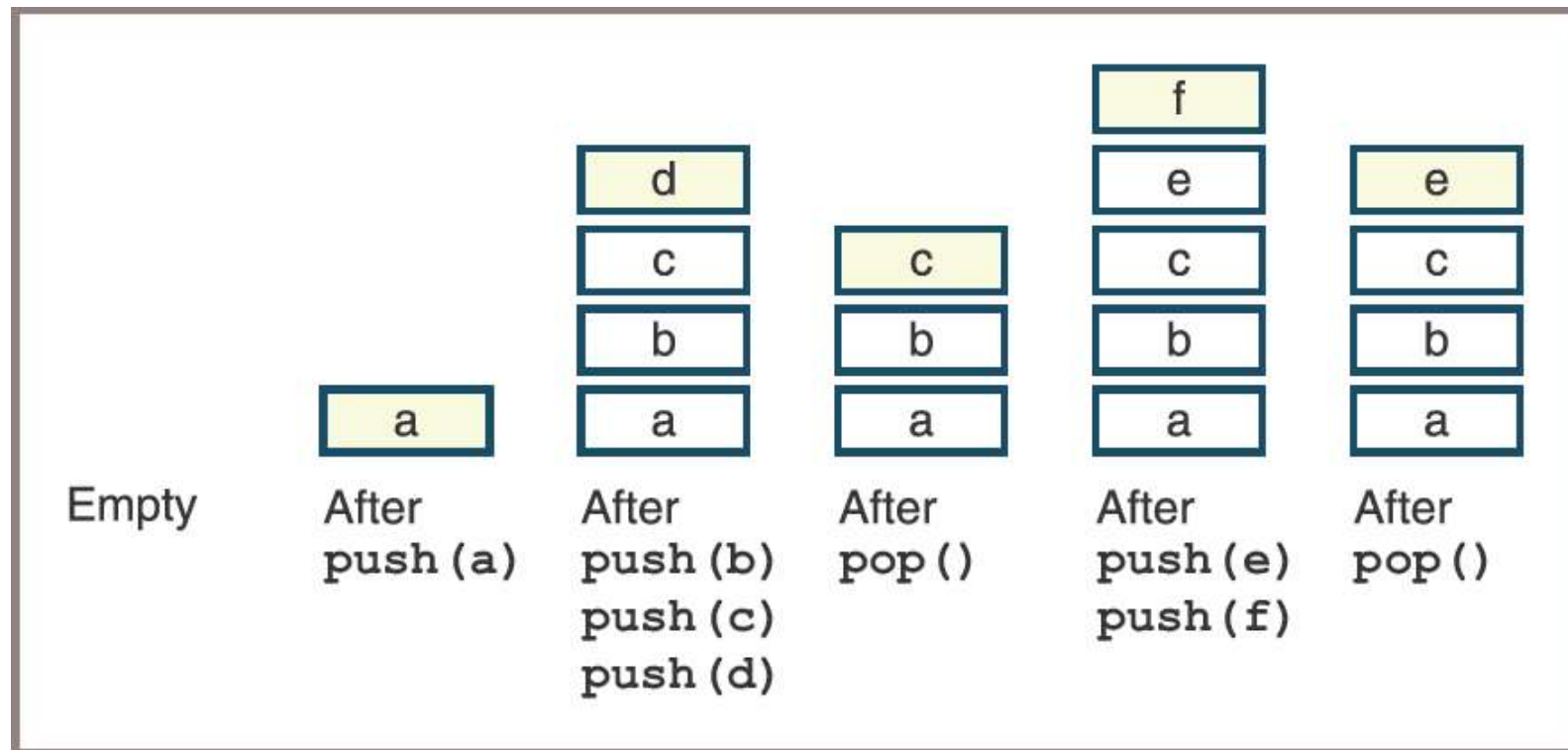
- Describe the features and behavior of a stack
- Choose a stack implementation based on its performance characteristics
- Recognize applications where it is appropriate to use a stack
- Explain how the system call stack provides run-time support for recursive subroutines
- Design and implement a backtracking algorithm that uses a stack

Overview of Stacks (1 of 3)

- Stacks – linear collections
 - Access is completely restricted to just one end called the *top*
 - Use a *last-in-first-out* (LIFO) protocol
- Analogy
 - Stack of clean trays in a cafeteria
 - New tray is removed from top of the stack, when needed
- Operations for putting items on and removing items from a stack are called **push** and **pop**

Overview of Stacks (2 of 3)

Figure 7-1: Some states in the lifetime of a stack



Overview of Stacks (3 of 3)

- Applications of stacks in computer science
 - Translating infix expressions to postfix form and evaluating postfix expressions
 - Backtracking algorithms
 - Managing computer memory in support of function and method calls
 - Supporting the undo feature in text editors, word processors, spreadsheet programs, drawing programs, and similar applications
 - Maintaining a history of the links visited by a web browser

Using a Stack

- A stack type is not built into Python
- Programmers can use a Python list to emulate an array-based stack:
 - The `list` method `append` pushes an element onto this stack
 - The `list` method `pop` removes and returns the element at its top
- The main drawback of this option is as follows:
 - All other list operations can manipulate the stack as well

The Stack Interface (1 of 5)

- A stack interface provides an operation named **peek** for examining the element at the top of a stack
- The stack type can also include operations such as
 - **clear**, **isEmpty**, **len**, **str**, **in**, and **+**
- These operations are listed as Python methods in Table 7.1 (see next slide)

The Stack Interface (2 of 5)

- Table 7.1 The Methods in the Stack Interface

Stack Method	What It Does
<code>s.isEmpty()</code>	Returns True if s is empty or False otherwise.
<code>s._len_()</code>	Same as <code>len(s)</code> . Returns the number of items in s .
<code>s._str_()</code>	Same as <code>str(s)</code> . Returns the string representation of s .
<code>s._iter_()</code>	Same as <code>iter(s)</code> , or <code>for item in s</code> . Visits each item in s , from bottom to top.
<code>s._contains_(item)</code>	Same as <code>item in s</code> . Returns True if item is in s or False otherwise.
<code>s1._add_(s2)</code>	Same as <code>s1 + s2</code> . Returns a new stack containing the items in s1 and s2 .

The Stack Interface (3 of 5)

- Table 7.1 The Methods in the Stack Interface (continued)

Stack Method	What It Does
<code>s._eq_(anyObject)</code>	Same as <code>s==anyObject</code> .
<code>s.clear()</code>	Makes <code>s</code> become empty.
<code>s.peek()</code>	Returns the item at the top of <code>s</code> . Precondition: <code>s</code> must not be empty; raises a KeyError if the stack is empty.
<code>s.push(item)</code>	Adds <code>item</code> to the top of <code>s</code> .
<code>s.pop()</code>	Removes and returns the item at the top of <code>s</code> . Precondition: <code>s</code> must not be empty; raises a KeyError if the stack is empty.

The Stack Interface (4 of 5)

- Table 7.2 The Effect of Stack Operations

Operation	State of Stack after the Operation	Value Returned	Comment
<code>s = <Stack Type> ()</code>			Initially, stack is empty.
<code>s.push (a)</code>	a		The stack contains the single item.
<code>s.push (b)</code>	ab		b is the top item.
<code>s.push (c)</code>	abc		c is the top item.
<code>s.isEmpty ()</code>	abc	False	The stack is not empty.
<code>len (s)</code>	abc	3	The stack contains three items.
<code>s.peak ()</code>	abc	c	Return the top item without removing it.

The Stack Interface (5 of 5)

- Table 7.2 The Effect of Stack Operations (continued)

Operation	State of Stack after the Operation	Value Returned	Comment
<code>s.pop ()</code>	<code>ab</code>	<code>c</code>	Remove and return the top item. <code>b</code> is now the top item.
<code>s.pop ()</code>	<code>a</code>	<code>b</code>	Remove and return the top item. <code>a</code> is now the top item.
<code>s.pop ()</code>		<code>a</code>	Remove and return the top item.
<code>s.isEmpty ()</code>		<code>True</code>	The stack is empty.
<code>s.peek ()</code>		<code>KeyError</code>	Peeking at an empty stack raises an exception.
<code>s.pop ()</code>		<code>KeyError</code>	Popping an empty stack raises an exception.
<code>s.push (d)</code>	<code>D</code>		<code>d</code> is the top item.

Instantiating a Stack

- Later in this chapter, two different implementations are considered:
 - **ArrayStack**
 - **LinkedStack**
- This code segment shows how you might instantiate them:
 - **s1=ArrayStack ()**
 - **s2=LinkedStack ([20, 40, 60])**

Example Application: Matching Parentheses (1 of 3)

- Compilers need to determine if the bracketing symbols in expressions are balanced correctly
- Table 7.3 provides some examples:
 - See next slide

Example Application: Matching Parentheses (2 of 3)

- Table 7.3 Balanced and Unbalanced Brackets in Expressions

Example Expression	Status	Reason
(...)...(...)	Balanced	
(...)....(...	Unbalanced	Missing a closing) at the end
)...(...(...)	Unbalanced	The closing) at the beginning has no matching opening (and one of the opening parentheses has no closing parenthesis
[...(...)...]	Balanced	
[...(...)...)	Unbalanced	The bracketed sections are not nested properly

Example Application: Matching Parentheses (3 of 3)

- To check an expression, take the following steps:
 - Scan across the expression, pushing opening brackets onto a stack
 - On encountering a closing bracket, if the stack is empty or if the item on the top of the stack is not an opening bracket of the same type, you know the brackets do not balance; therefore, you can quit the process and signal that the expression is improperly formed
 - Otherwise, pop the item off the top of the stack and continue scanning the expression
 - When you reach the end of the expression, the stack should be empty, and if it is not, you know the brackets do not balance

Three Applications of Stacks

- First, you'll see algorithms for evaluating arithmetic expressions:
 - These algorithms apply to problems in compiler design, and you will use them in the chapter's case study
- Second, you'll learn a general technique for using stacks to solve backtracking problems:
 - The programming projects explore applications of the technique
- Third, you'll witness the role of stacks in computer memory management

Evaluating Arithmetic Expressions (1 of 3)

- An indirect approach to evaluating arithmetic expressions works best:
 - Transform an expression from its familiar infix form to a postfix form
 - Evaluate the postfix form
 - In the infix form, each operator is located between its operands
 - In the postfix form, an operator immediately follows its operands

Evaluating Arithmetic Expressions (2 of 3)

- Table 7.4 Some Infix and Postfix Expressions

Infix Form	Postfix Form	Value
34	34	34
34 + 22	34 22 +	56
34 + 22 * 2	34 22 2*+	78
34 * 22 + 2	34 22 * 2 +	750
(34 + 22) * 2	34 22 + 2 *	112

Evaluating Arithmetic Expressions (3 of 3)

- Consider the steps in evaluating the following:
 - Infix evaluation: $34 + 22 * 2 \rightarrow 34 + 44 \rightarrow 78$
 - Postfix evaluation: $34 \ 22 \ 2 \ * \ + \rightarrow 34 \ 44 \ + \rightarrow 78$
- The use of parentheses and operator precedence in infix expressions is for the convenience of the human beings who read them and write them
- By eliminating these parentheses,
 - The equivalent postfix expressions present a computer with a format that is much easier and more efficient for it to evaluate

Evaluating Postfix Expressions (1 of 3)

- Evaluating a postfix expression involves three steps:
 - Scan across the expression from left to right
 - On encountering an operator, apply it to the two preceding operands and replace all three by the result
 - Continue scanning until you reach the expression's end, at which point only the expression's value remains

Evaluating Postfix Expressions (2 of 3)

- Table 7.5 Tracing the Evaluation of a Postfix Expression

Postfix Expression: 4 5 6 * +3 –		Resulting Value: 31
Portion of Postfix Expression Scanned So Far	Operand Stack	Comment
		No tokens have been scanned yet, so the stack is empty
4	4	Push the operand 4
4 5	4 5	Push the operand 5
4 5 6	4 5 6	Push the operand 6
4 5 6 *	4 30	Replace the top two operands by their product

Evaluating Postfix Expressions (3 of 3)

- Table 7.5 Tracing the Evaluation of a Postfix Expression (continued)

Postfix Expression: 4 5 6 * + 3 -		Resulting Value: 31
Portion of Postfix Expression Scanned So Far	Operand Stack	Comment
4 5 6 * +	34	Replace the top two operands by their sum
4 5 6 * + 3	34 3	Push the operand 3
4 5 6 * + 3 -	31	Replace the top two operands by their difference and pop the final value

Converting Infix to Postfix (1 of 3)

- Detailed steps
 - Start with an empty postfix expression and an empty stack, which will hold operators and left parentheses
 - Scan across the infix expression from left to right
 - On encountering an operand, append it to the postfix expression
 - On encountering a left parenthesis, push it onto the stack
 - On encountering an operator, pop off the stack all operators that have equal or higher precedence, append them to the postfix expression, and then push the scanned operator onto the stack
 - On encountering a right parenthesis, shift operators from the stack to the postfix expression until meeting the matching left parenthesis, which is discarded
 - On encountering the end of the infix expression, transfer the remaining operators from the stack to the postfix expression

Converting Infix to Postfix (2 of 3)

- Table 7.6 Tracing the Conversion of an Infix Expression to a Postfix Expression

Infix Expression: 4 + 5 * 6 – 3 Portion of Infix Expression Scanned So Far	Operator Stack	Postfix Expression: 4 5 6 * + 3 –	Comment
			No tokens have been scanned yet, so the stack and PE are empty
4		4	Append 4 to the PE
4 +	+		Push + onto the stack
4 + 5	+	4 5	Append 5 to the PE
4 + 5 *	+ *	4 5	Push * onto the stack
4 + 5 * 6	+ *	4 5 6	Append 6 to the PE

Converting Infix to Postfix (3 of 3)

- Table 7.6 Tracing the Conversion of an Infix Expression to a Postfix Expression (continued)

Infix Expression: 4 + 5 * 6 - 3 Portion of Infix Expression Scanned So Far	Operator Stack	Postfix Expression: 4 5 6 * + 3 -	Comment
4 + 5 * 6 -	-	4 5 6 * +	+ Pop * and +, append them to the PE, and push - onto the stack
4 + 5 * 6 - 3	-	4 5 6 * + 3	Append 3 to the PE
4 + 5 * 6 - 3		4 5 6 * + 3	Pop the remaining operators off the stack and append them to the PE

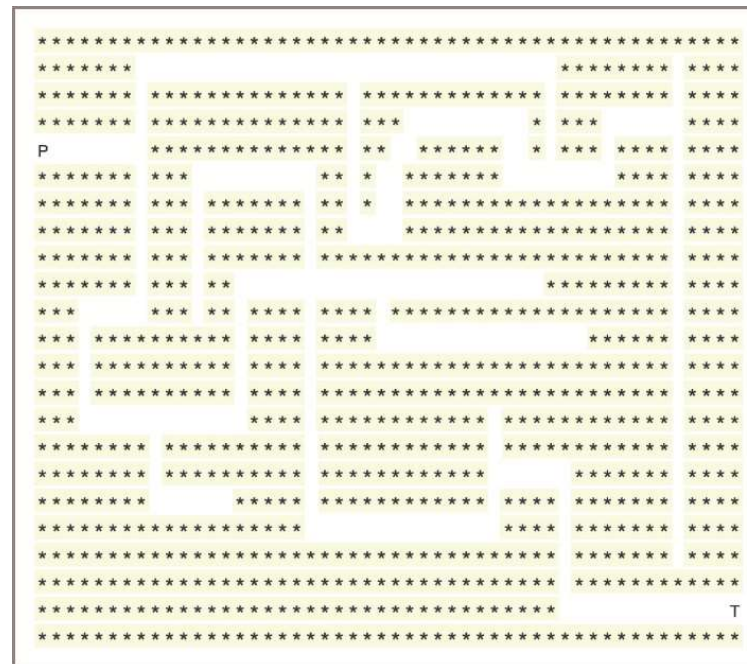
Backtracking (1 of 3)

- Backtracking algorithm
 - Begins in a predefined starting state and moves from state to state in search of a desired ending state
 - If the algorithm reaches a state that represents an undesirable outcome, it backs up to the last point at which there was an unexplored alternative and tries it
- Two principal techniques for implementing backtracking algorithms
 - One uses stacks
 - The other uses recursion

Backtracking (2 of 3)

- The role of a stack in the process is to remember the alternative states that occur at each juncture:
 - Consider the problem of finding a path out of a maze

Figure 7-2: A maze problem



Backtracking (3 of 3)

- Sample backtracking algorithm:

Create a new stack

Locate the character 'P' in the grid

Push its location onto the stack

While the stack is not empty

 Pop a location, (row, column), off the stack

 If the grid contains 'T' at this location, then

 A path has been found

 Return True

 Else if this location does not contain a dot

 Place a dot in the grid at this location

 Examine the adjacent cells to this one and

 for each one that contains a space,

 push its location onto the stack

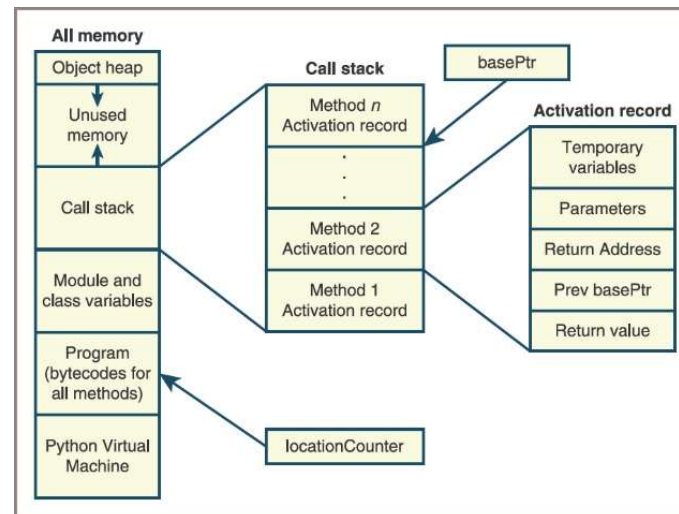
Return False

Memory Management (1 of 6)

- During a program's execution, both its code and its data occupy computer memory
- The computer's run-time system must keep track of various details that are invisible to the program's author, which include the following:
 - Associating variables with data objects stored in memory so they can be located when these variables are referenced
 - Remembering the address of the instruction in which a method or function is called, so control can return to the next instruction when that function or method finishes execution
 - Allocating memory for a function's or a method's arguments and temporary variables, which exist only during the execution of that function or method

Memory Management (2 of 6)

- A Python compiler translates a Python program into bytecodes:
 - A complex program called the Python Virtual Machine (PVM) then executes these
- The memory, or *run-time environment*, controlled by the PVM is divided into six regions
- Figure 7-3: The architecture of a run-time environment



Memory Management (3 of 6)

- The activation records shown in the figure contain two types of information:
 - The regions labeled Temporary Variables and Parameters hold data needed by the executing subroutine
 - The remaining regions hold data that allow the PVM to pass control backward from the currently executing subroutine to the subroutine that called it

Memory Management (4 of 6)

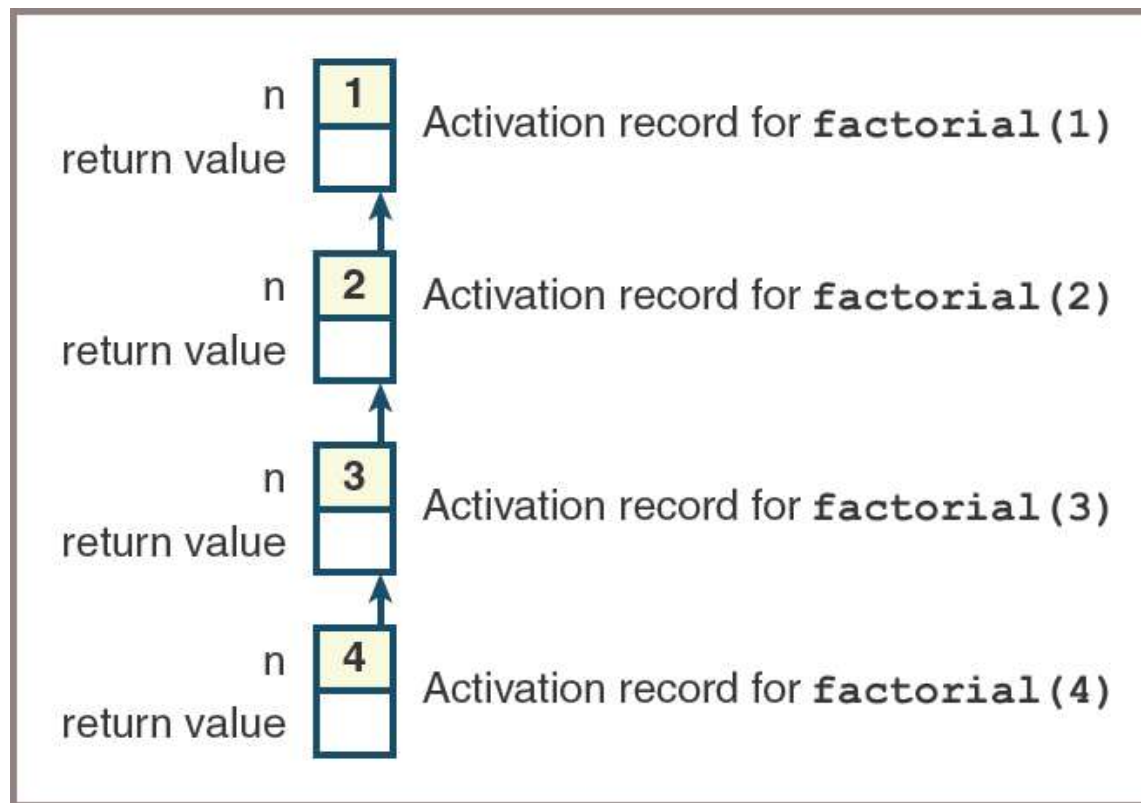
- When a subroutine is called, the PVM performs the following steps:
 - Creates the subroutine's activation record and pushes it onto the call stack
 - Saves the **basePtr**'s current value in the region labeled **Prev basePtr** and sets the **basePtr** to the new activation record's base
 - Saves the **locationCounter**'s current value in the region labeled **Return Address** and sets the **locationCounter** to the first instruction of the called subroutine
 - Copies the calling parameters into the region labeled **Parameters**
 - Starts executing the called subroutine at the location indicated by the **locationCounter**

Memory Management (5 of 6)

- When a subroutine has finished executing, the PVM performs the following steps:
 - Reestablishes the settings needed by the calling subroutine by restoring the values of the **locationCounter** and the **basePtr** from values stored in the activation record
 - Pops the activation record from the call stack
 - Resumes execution of the calling subroutine at the location indicated by the **locationCounter**

Memory Management (6 of 6)

Figure 7-4: The call stack needed to execute `factorial(4)`



Implementations of Stacks

- Stacks are implemented easily using arrays or linked structures
- The following sections show that there are trade-offs involved in using these two recurring approaches

Test Driver

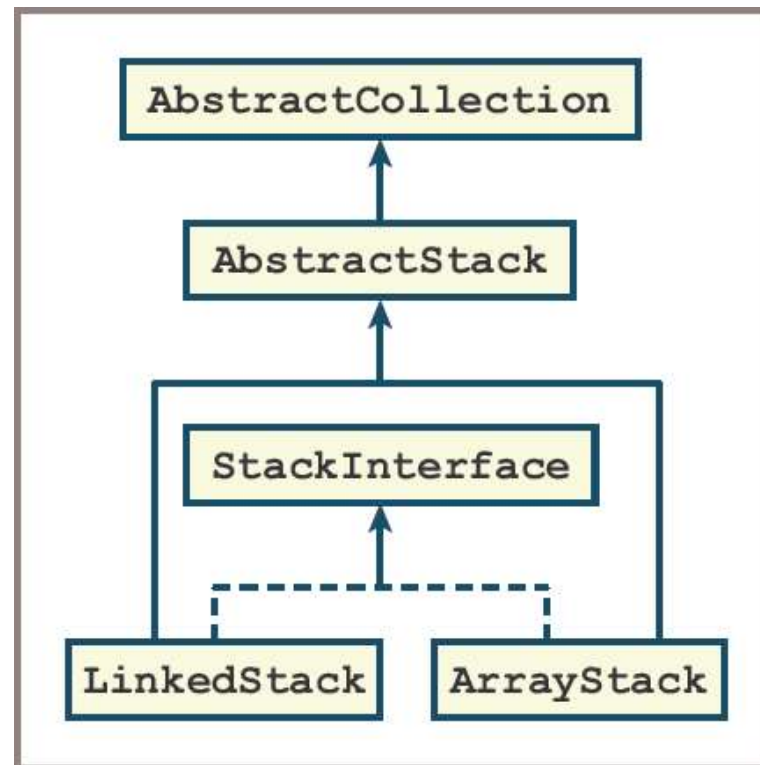
- Two stack implementations are the classes **ArrayStack** and **LinkedStack**
- Before you develop these, write a short tester program that shows how you can test them immediately
- The code in this program exercises all the methods in any stack implementation and gives you an initial sense that they are working as expected:
 - See the text for the code along with the output

Adding Stacks to the Collection Hierarchy (1 of 2)

- **ArrayStack** and **LinkedStack**
 - Implement the same interface, called **StackInterface**
 - Are subclasses of the **AbstractStack** class, which in turn is a subclass of **AbstractCollection**
 - Inherit the **add** method from the **AbstractStack** class, the **size** variable, and the methods **isEmpty**, **__len__**, **__str__**, **__add__**, and **__eq__** from **AbstractCollection**
- The only methods that need to be implemented in **ArrayStack** and **LinkedStack** are **_init_**, **peek**, **push**, **pop**, **clear**, and **_iter_**

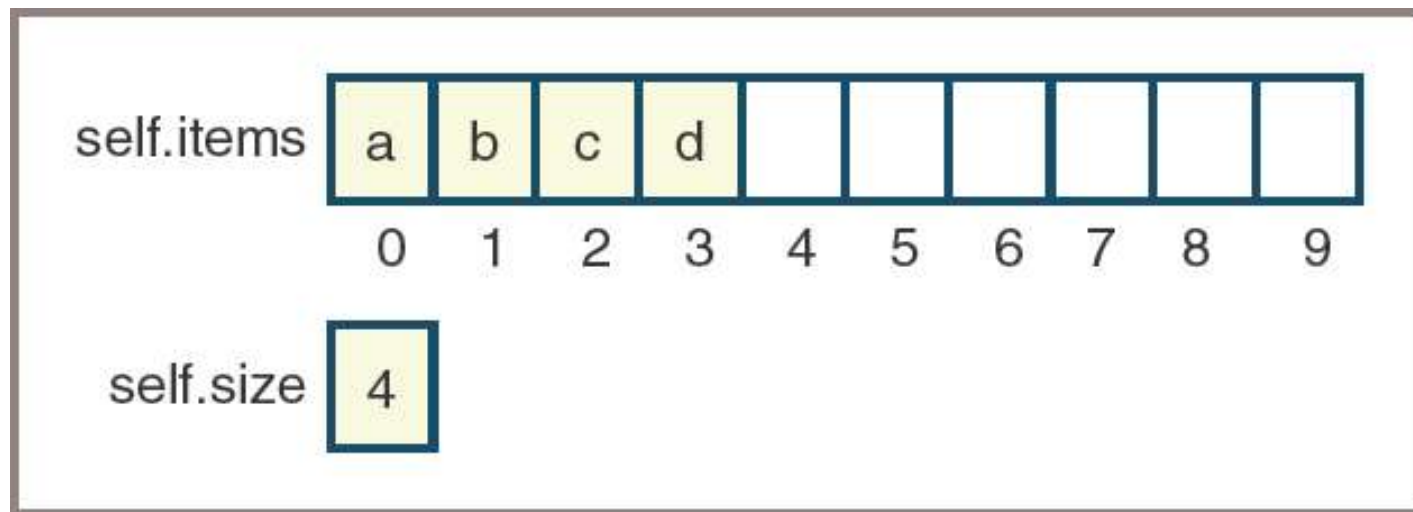
Adding Stacks to the Collection Hierarchy (2 of 2)

Figure 7-5: The stack resources in the collection hierarchy



Array Implementation

- The first implementation is built around an array called **`self.items`** and an integer called **`self.size`**
- Figure 7.6 shows how **`self.items`** and **`self.size`** appear when four items are on the stacks
- Figure 7-6: An array representation of a stack with four items



Linked Implementation (1 of 4)

- The linked implementation requires two classes:
 - **LinkedStack** and **Node**
- The Node class contains two fields:
 - **data** — An item on the stack
 - **next** — A pointer to the next node
- Because new items are added to and removed from just one end of the linked structure, the methods `pop` and `push` are easy to implement, as shown in Figures 7.8 and 7.9

Linked Implementation (2 of 4)

Figure 7-8: Pushing an item onto a linked stack

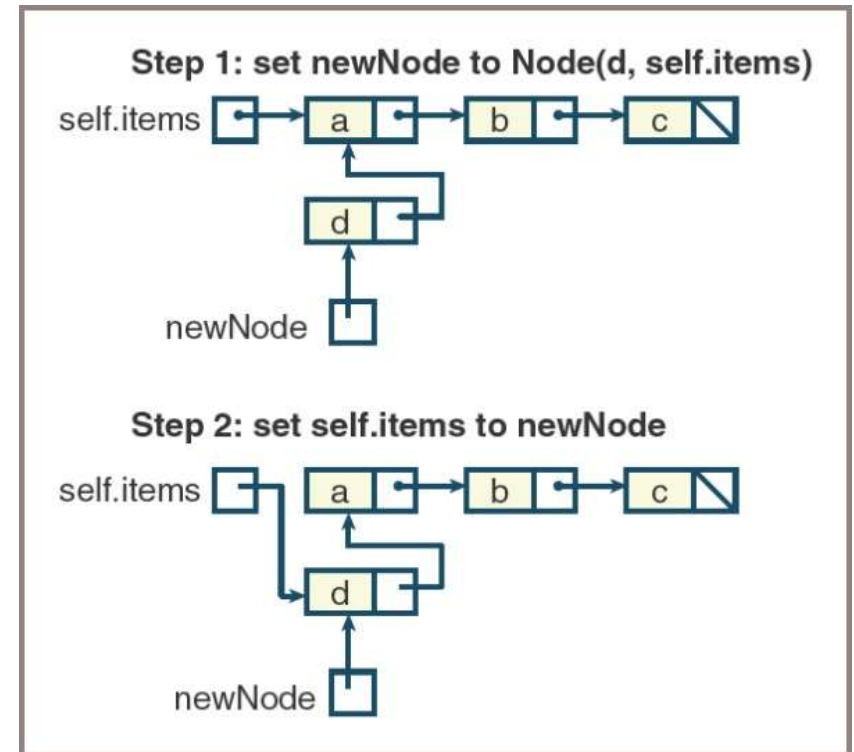
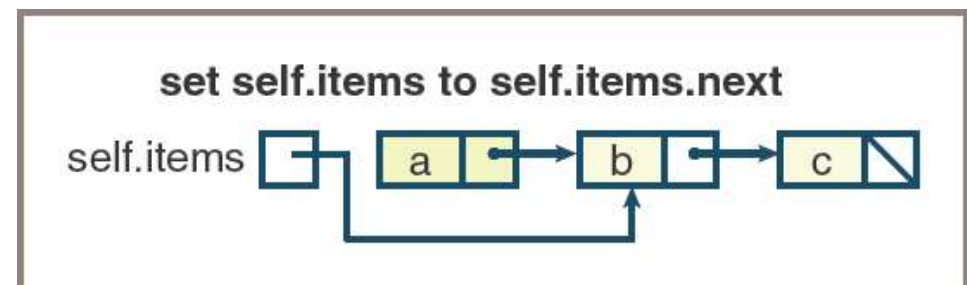


Figure 7-9: Popping an item from a linked stack



Linked Implementation (3 of 4)

- Code for **LinkedStack**:

```
from node import Node
from abstractstack import AbstractStack

class LinkedStack(AbstractStack):
    """Link-based stack implementation."""

    def __init__(self, sourceCollection = None):
        self.items = None
        AbstractStack.__init__(self, sourceCollection)

    # Accessors
    def __iter__(self):
        """Supports iteration over a view of self.
        Visits items from bottom to top of stack."""
        def visitNodes(node):
            if node != None:
                visitNodes(node.next)
                tempList.append(node.data)
        tempList = list()
        visitNodes(self.items)
        return iter(tempList)

    def peek(self):
        """Returns the item at top of the stack.
        Precondition: the stack is not empty."""
        if self.isEmpty():
            raise KeyError("The stack is empty.")
        return self.items.data
```

Linked Implementation (4 of 4)

- Code for `LinkedStack` (continued):

```
# Mutators
def clear(self):
    """Makes self become empty."""
    self.size = 0
    self.items = None

def push(self, item):
    """Inserts item at top of the stack."""
    self.items = Node(item, self.items)
    self.size += 1

def pop(self):
    """Removes and returns the item at top of the stack.
    Precondition: the stack is not empty."""
    if self.isEmpty():
        raise KeyError("The stack is empty.")
    oldItem = self.items.data
    self.items = self.items.next
    self.size -= 1
    return oldItem
```

The Role of the Abstract Stack Class (1 of 2)

- The `__init__` method in **AbstractCollection** uses the `add` method to add items in a source collection to **self**:
 - If **self** is a stack, Python raises an exception, stating that the `add` method is undefined for stacks
- To remedy this problem and maintain consistency with the interfaces of other collections,
 - You need to include an `add` method with your stack types
 - The logical place to put this method, so that all stack types can use it, is in the **AbstractStack** class

The Role of the Abstract Stack Class (2 of 2)

- Code for AbstractStack:

```
"""
File: abstractstack.py
Author: Ken Lambert
"""

from abstractcollection import AbstractCollection

class AbstractStack(AbstractCollection):
    """An abstract stack implementation."""

    # Constructor
    def __init__(self, sourceCollection):
        """Sets the initial state of self, which includes the
        contents of sourceCollection, if it's present."""
        AbstractCollection.__init__(self, sourceCollection)

    # Mutator methods
    def add(self, item):
        """Adds item to self."""
        self.push(item)
```

Time and Space Analysis of the Two Implementations (1 of 2)

- With the exception of the `__iter__` method,
 - All the stack methods are simple and have a maximum running time of $O(1)$, or constant
- In the array implementation, the analysis becomes more complex:
 - At the moment of doubling, the `push` method's running time jumps to $O(n)$, or linear, but the rest of the time it remains at $O(1)$
 - Similar remarks can be made about the `pop` method
 - On average, both are still $O(1)$
- The recursive function used in the linked implementation causes a linear growth of memory because of its use of the system call stack:
 - Avoid this problem by using a doubly linked structure; the iterator can begin at the last node and follow links to previous nodes

Time and Space Analysis of the Two Implementations (2 of 2)

- A linked stack of n items requires n nodes, each containing two references:
 - One to an item and the other to the next node
 - In addition, there must be a variable that points to the top node and a variable for the size, yielding a total space requirement of $2n + 2$
- For an array implementation,
 - Assuming that an integer and a reference occupy the same amount of space, the total space requirement is the array's capacity + 2
- An array implementation is more space-efficient than a linked implementation whenever the load factor is greater than $\frac{1}{2}$

Chapter Summary (1 of 2)

- A stack is a linear collection that allows access to one end only, called the top:
 - Elements are pushed onto the top or popped from it
- Other operations on stacks include peeking at the top element, determining the number of elements, determining whether the stack is empty, and returning a string representation
- Stacks are used in applications that manage data items in a last-in-first-out manner
- These applications include the following:
 - Matching bracket symbols in expressions, evaluating postfix expressions, backtracking algorithms, and managing memory for subroutine calls on a virtual machine

Chapter Summary (2 of 2)

- Arrays and singly linked structures support simple implementations of stacks